

# AULA 1

## Introdução ao Framework e Arquitetura





**PROFESSOR**

## **Dr. Fernando Antônio Mota Trinta**

Departamento de Computação — Universidade Federal do Ceará (UFC)

- Responsável pela disciplina CK0207 — Desenvolvimento de Software para Web



# OBJETIVOS DA AULA



AO FINAL DESSA AULA, VOCÊ VAI SER CAPAZ DE...

- Entender por que o NestJS foi criado
- Identificar problemas comuns do Express em aplicações grandes
- Compreender a arquitetura do Nest
- Entender o papel de Modules, Controllers e Services
- Compreender Injeção de Dependências
- Criar um projeto utilizando a Nest CLI



# MOTIVAÇÃO PARA O NESTJS



## O QUE O NESTJS TRAZ PRA MESA

- Arquitetura opinativa, fortemente inspirada no Angular
- Injeção de Dependências (DI) nativa — o framework cria e entrega suas classes
- Módulos, Controllers e Services bem definidos desde o início
- Roda por cima do Express (padrão) ou do Fastify — não perde nada do ecossistema Node
- Ganho prático: mais testabilidade e mais facilidade pra escalar o projeto com o time



# LIMITAÇÕES DO EXPRESS EM PROJETOS GRANDES



## POR QUE ISSO PODE VIRAR UM PROBLEMA

- Express dá roteamento e middlewares, mas não impõe nenhuma estrutura
- Sem padrão de organização entre times/desenvolvedores
- Injeção de dependência manual (ou inexistente) — "new Service()" espalhado pelo código
- Sem separação clara entre camada HTTP e regra de negócio
- Com disciplina isso não é regra — mas sem ela, vira controllers gigantes e baixa testabilidade



# ARQUITETURA DO NESTJS



## FLUXO DE UMA REQUISIÇÃO

**Cliente** → Controller (*HTTP*) → Service (*regra de negócio*) → Banco / API externa → Resposta

## REGRA DE OURO

O controller nunca fala diretamente com o banco de dados — ele sempre passa pelo service.

Essa separação é o que vai permitir, mais pra frente no curso, trocar a forma como persistimos dados ou testar a lógica de negócio isoladamente.



# CONTROLLERS E SERVICES



## CONTROLLERS

- Recebem a requisição HTTP e devolvem a resposta
- Não devem conter lógica de negócio
- Delegam qualquer trabalho mais complexo para um service
- Muitos "if", cálculo ou validação manual aqui é sinal de que essa lógica devia estar em outro lugar

## SERVICES (PROVIDERS)

- Classes simples marcadas com `@Injectable()`
- É aqui que mora a regra de negócio de verdade
- Processamento de dados, acesso ao banco, chamadas externas
- O Nest gerencia o ciclo de vida: por padrão, uma instância única (singleton) reutilizada por toda a aplicação



# MODULES



## @MODULE() — AS 4 PROPRIEDADES

```
@Module({
  controllers: [UsersController],
  providers: [UsersService],
  imports: [OtherModule],
  exports: [UsersService],
})
export class UserModule {}
```

*controllers, providers, imports e exports — cada um com um papel específico no módulo.*

## MÓDULO RAIZ E MÓDULOS DE FEATURE

- Um módulo agrupa controllers e providers de um mesmo domínio
- Toda aplicação tem pelo menos um módulo raiz — o AppModule
- A partir dele, o projeto cresce criando módulos de feature: UsersModule, AuthModule, QuestionModule...
- Cada módulo é uma "caixinha" que empacota tudo que pertence àquele domínio





# INJEÇÃO DE DEPENDÊNCIAS — O CONCEITO



## INVERSÃO DE CONTROLE (IoC)

```
@Injectable()
export class UsersService {}

@Controller('users')
export class UsersController {
  constructor(
    private usersService: UsersService,
  ) {}
}
```

*O Nest cria e entrega a instância de UsersService automaticamente — você não escreve "new UsersService()".*

## POR QUE ISSO IMPORTA NA PRÁTICA

- Testabilidade: dá pra trocar um service real por um mock sem tocar no controller
- Desacoplamento: o controller só precisa saber o tipo, não como o service é construído
- Fica mais fácil quando vários services passam a depender uns dos outros
- O Nest tem um container IoC embutido que resolve tudo isso usando os tipos do TypeScript



# INJEÇÃO DE DEPENDÊNCIAS — POR TRÁS DOS PANOS



## 3 PASSOS DO REGISTRO

- 1. `@Injectable()` marca a classe como gerenciável pelo container IoC
- 2. O Controller declara a dependência no construtor (injeção via construtor)
- 3. O Module associa ("registra") o token à classe em providers: `[UserService]`

## CICLO DE VIDA

Por padrão, cada provider é um singleton: instanciado uma vez no bootstrap e reutilizado por toda a aplicação até ela encerrar. Existe a opção de escopo por requisição, mas isso é um tópico mais avançado.



# NEST CLI E ESTRUTURA DE PROJETOS



## COMANDOS ESSENCIAIS

```
npm install -g @nestjs/cli  
  
nest new nome-do-projeto  
  
nest generate module users    (nest g mo users)  
nest generate controller users (nest g co users)  
nest generate service user    (nest g s users)
```

## ESTRUTURA POR DOMÍNIO/FEATURE

```
src/  
  users/  
    dto/  
      create-users.dto.ts  
    users.controller.ts  
    users.service.ts  
    users.module.ts  
  app.module.ts  
  main.ts
```

*Cada domínio isolado facilita localizar código e reduz acoplamento entre partes do sistema.*

